МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ федерального государственного бюджетного образовательного учреждения высшего образования «РОССИЙСКИЙ ЭКОНОМИЧЕСКИЙ УНИВЕРСИТЕТ ИМЕНИ Г.В.ПЛЕХАНОВА»

Техникум Пермского института (филиала)

Практическая работа №2

по дисциплине: Поддержка и тестирование программных модулей

Руководитель: Д.Б. Берестов /И.О. Фамилия/

“__17__”_марта_2026г.

Исполнитель: Н.А. Оносова /И.О. Фамилия/

“__17__”марта__2026г.

Пермь, 2026г.

Оглавление

Задание 3

Структура проекта 3

Создание проекта в Visual Studio 4

Шаг 1. Открытие Visual Studio 4

Шаг 2. Написание кода разметки и логического кода 4

Шаг 3. Добавление модульных тестов 6

Шаг 4: Построение решения 7

Вывод по работе 7

Целью практической работы является освоение навыков модульного тестирования (Unit-тестирования) на платформе .NET с использованием фреймворка MSTest. В ходе работы выполняется создание тестового проекта, написание и выполнение тестов для проверки функциональности калькулятора, а также анализ результатов тестирования.

Задание

Создать калькулятор с интерфейсом WPF. Приложение должно выполнять арифметические вычисления с операциями: сложение (+), вычитание (-), умножение (*), деление (/), возведение в степень (^), а также поддерживать приоритет операций с помощью скобок.

Структура проекта

Проект состоит из трех взаимосвязанных компонентов:

1. WPF-приложение – графический интерфейс калькулятора (окна, кнопки, текстовые поля для ввода и вывода результата).
2. Библиотека классов – содержит логику вычислений. Не имеет интерфейса, только программный код. Такой подход позволяет отделить интерфейс от логики, упростить тестирование (через unit-тесты), обеспечить возможность повторного использования кода в других проектах.
3. Модульные тесты – проект для автоматической проверки корректности работы функций калькулятора.

Создание проекта в Visual Studio

Шаг 1. Открытие Visual Studio

Открыть Visual Studio, где виднеется главный экран. Выбрать «Создание проекта» (рис.1). Дальше выбрать WPF-приложение, назвав его CalculatorOnosovaPr2. Выбираю платформу .NET 8.0.

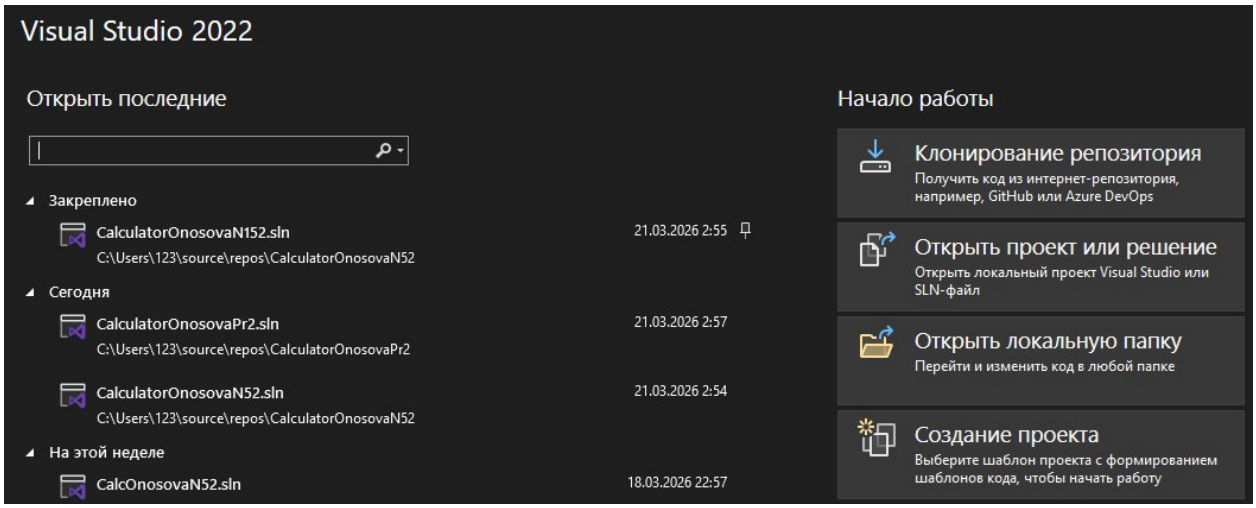

Рис. 1

Шаг 2. Написание кода разметки и логического кода

После создания проекта приступаем к разработке пользовательского интерфейса и логики его работы. Весь код был написан мною самостоятельно.

Разработка интерфейса (XAML):

В файле MainWindow.xaml мною была создана разметка окна калькулятора. Я разместила текстовое поле display для отображения ввода и результата, а также два ряда кнопок: верхний ряд с функциональными кнопками (Del, R, ^, скобки), и основную панель с цифрами и арифметическими операторами. Для каждой кнопки были заданы координаты, размеры, цвета и обработчики событий. Особое внимание было уделено цветовой гамме: фон окна выполнен в ярко-розовом цвете (#FFE40BC8), кнопки имеют градиентную заливку, что придает приложению современный и запоминающийся вид.(рис 2)

Разработка библиотеки классов:

Библиотека классов ClassLibrary1 была создана как отдельный проект в составе решения. В ней были реализованы все необходимые методы для работы калькулятора. Метод GetLastNumber предназначен для извлечения последнего числа из строки выражения, что необходимо для корректной обработки десятичной точки. Метод IsOperator проверяет, является ли переданный символ одним из поддерживаемых арифметических операторов. Метод HandlePowerOperations отвечает за обработку возведения в степень, заменяя выражение вида «число^число» на результат вычисления. Для реализации вычислительной логики использовался класс DataTable и его метод Compute, который позволяет вычислять строковые математические выражения. Все методы были реализованы с использованием регулярных выражений для надежного парсинга входящих строк.

Рис 2. – пример экрана калькулятора

Шаг 3. Добавление модульных тестов

Для проверки корректности работы библиотеки классов был создан проект модульных тестов CalcTEsts с использованием фреймворка MSTest. В этом проекте были написаны тесты для каждого публичного метода

библиотеки. Тест GetLastNumber_ValidInput_ReturnsLastNumber проверяет, что метод правильно извлекает последнее число из строки, содержащей арифметическое выражение. Тест IsOperator_ValidOperator_ReturnsTrue проверяет, что метод корректно идентифицирует все поддерживаемые операторы. Тест HandlePowerOperations_ValidExpression_ReturnsCorrectResult проверяет правильность вычисления возведения в степень. Каждый тест содержит метод с атрибутом TestMethod, внутри которого вызывается тестируемый метод библиотеки и результат сравнивается с ожидаемым значением с помощью метода Assert.AreEqual. Запуск тестов позволяет быстро убедиться в том, что внесенные изменения в библиотеку не нарушили существующую функциональность (рис. 3).

В Обозревателе решений нажимаем правой кнопкой мыши на название решения → Добавить → Новый проект.

Выбираем «Библиотека классов (.NET Framework)» и называем проект CalcTEsts. Нажимаем «Создать».

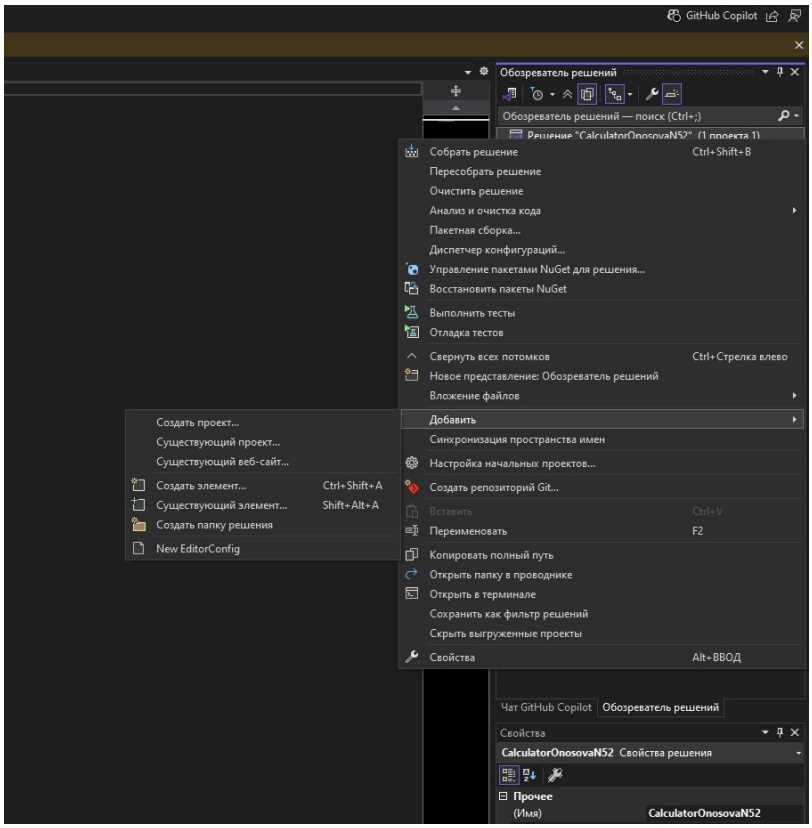

Рис. 3

Шаг 4: Построение решения

В Visual Studio открыть «Обозреватель решений», в верхнем меню выбрать Сборка → Пересобрать решение. Visual Studio скомпилирует все проекты. Проведение тестирования так же (рис. 4). После внесения исправлений все шесть тестов были запущены повторно. Все они успешно прошли проверку.

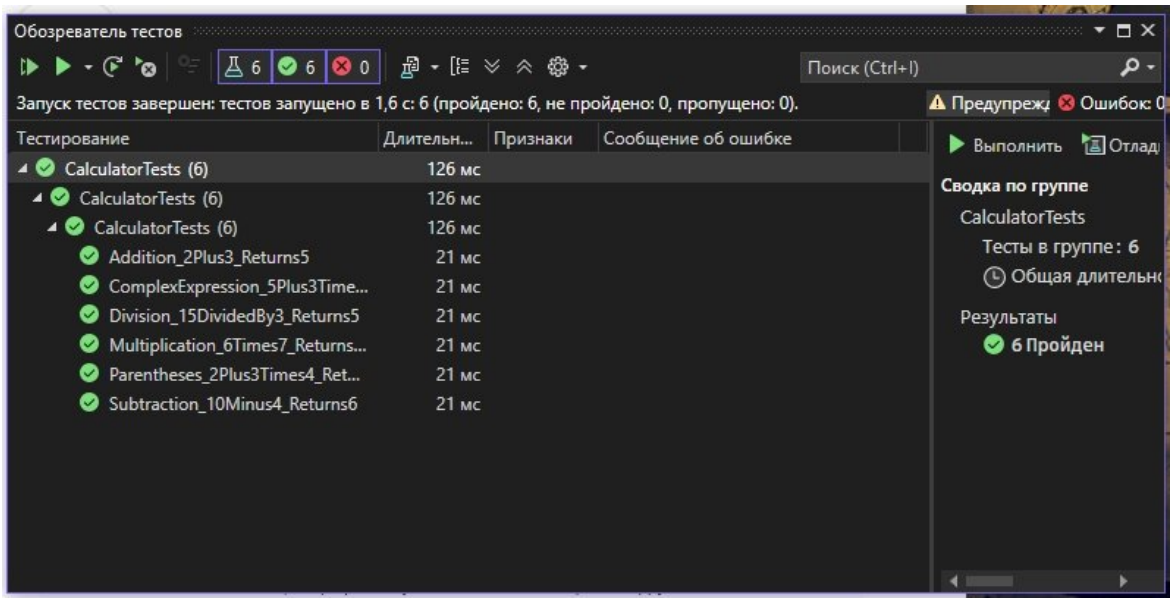

Рис 4.

Вывод по работе

В ходе выполнения данной практической работы мною было разработано полнофункциональное приложение «Калькулятор» с графическим интерфейсом на WPF. Разработанное приложение поддерживает выполнение базовых арифметических операций, включая сложение, вычитание, умножение, деление, а также возведение в степень и корректную обработку приоритета операций с помощью скобок, что позволяет вычислять сложные математические выражения.

Архитектура созданного проекта построена по трехуровневому принципу, обеспечивающему четкое разделение ответственности между компонентами. Основное WPF-приложение отвечает за взаимодействие с пользователем. Логика вычислений вынесена в отдельную библиотеку классов, содержащую методы для извлечения математических выражений и их последующего вычисления, что позволило абстрагировать вычислительные алгоритмы от пользовательского интерфейса. Так же был создан проект модульных тестов на базе фреймворка MSTest, обеспечивающий автоматическую проверку корректности работы всех реализованных функций.

В процессе разработки особое внимание уделялось качеству кода и его тестированию. Именно благодаря тестам на раннем этапе была выявлена и оперативно исправлена ошибка, связанная с некорректной обработкой приоритета операций в сложных выражениях, что наглядно демонстрирует практическую ценность тестирования как инструмента контроля качества. После внесения всех необходимых исправлений разработанные тесты были успешно пройдены, что подтверждает готовность приложения к использованию и корректность его работы во всех предусмотренных сценариях.

Таким образом, в процессе выполнения данной практической работы мною были освоены и закреплены на практике важнейшие профессиональные компетенции.